

# Writing Simple Python Programs

CMSC 104 Section 02 Spring 2026  
February 4, 2026

# Administrative Notes

Classwork 1 was assigned on Monday

- It must be submitted via Blackboard
- It is due next Monday - February 9 - BEFORE midnight

Classwork 2 will be assigned today

- It is due next Wednesday - February 11 - BEFORE midnight

# A bit about how computers work internally

You can think of the computer's memory as a long set of places to store bits

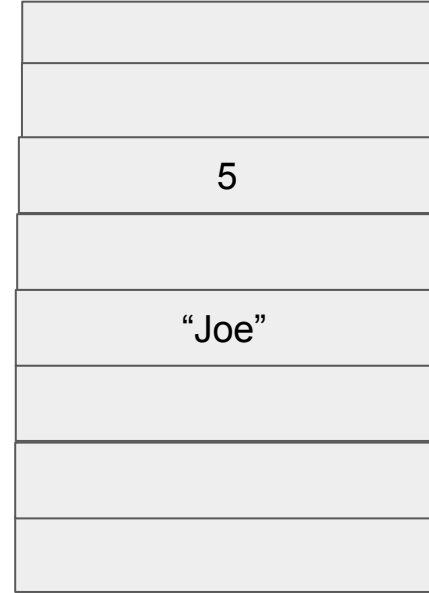
- Each place has an internal address (that we don't care about - it depends on what else is running on your computer at the time)
- How many bits can be stored depends on your computer; 32 and 64 are typical
- What those bits mean depend on what type of data we're talking about

140733399356328

5

140733399356344

"Joe"



# Python helps us manage this

Python creates a “symbol table” with every identifier (variable name) you use in your program

- It allocates a location in memory
- Identifies what type it's supposed to hold (integer, real number, string,...)
- Maps your name to the location's address

When you use that variable in your program, Python goes to that address in memory and gets the value



# A simplified illustration

| Symbol` | Type    | Address         |
|---------|---------|-----------------|
| index   | integer | 140733399356328 |
| name    | string  | 140733399356344 |
|         |         |                 |

The fact that Python manages this means we don't have to!!!

# What's a “symbol?”

Okay, we have a “symbol table”. What's a “symbol?”

- It's any name you use in your program
- Right now, we've only learned about “variables”
  - There will be more later
- Why is it a “variable?”
  - Because its value can change in the program
- What are the rules on variable names, again?
  - Start with an underscore ( ) or letter - upper or lower case are both fine
  - Contains underscores, upper & lower case letters, and digits
  - Can be as long as you want



# Strings, Reals and Integers

# The computer only uses bits...

Internally, computers only understand collections of 1's and 0's

But high-level programmers (like us) need the computer to understand things like real numbers, integer numbers, letters, groups of letters, lists of things, ...

- Because that's what we understand

So the solution is for the system to understand the “type” of each data item

# What does 1000001 mean?

Suppose a memory location in the computer stores the series of bits 1000001

What does that mean?

- By convention, that's how you represent an uppercase "A"
- But when you convert 1000001 from base 2 into base 10, you get the integer number 65

So which do you mean - "A" or 65?

Python resolves this by giving every value and variable a "type"

The simplest types are strings, integer numbers and real numbers

# Types

We'll get to integer and real numbers mostly next week

Strings: a string is a set of 0 or more characters that is treated as a unitary item

- Characters can be anything that can be represented on your computer
  - Digits; upper & lower case characters; special characters; whitespace like blank spaces, tabs and newline characters - they're all valid

What can you do with a string?

Lots of things, but relevant to classwork 2: you can concatenate them

- `string1+string2` yields `string1string2`
  - The two strings combined in the specified order with no spaces between them
  - If you want a space between them you have to explicitly add it
    - `string1 + " " + string2`

# More on Print statements

# Printing strings

To print a literal string, put it between quotes:

```
print("Hello, world")
```

To print a variable that holds a string, just use the variable name:

```
print(name)
```

# Printing multiple values in a single statement

You can print multiple values in the same print statement

- If you separate the values with a + they will be concatenated - no space between them
- If you separate the values with a comma , there will be one blank space printed between them

```
print("hello, world"+name)
```

```
print("hello, world",name)
```

# Input Statements

# Getting data from the user

At this point in the class, all data comes from the user via the keyboard.

You do that using the “input” statement

The syntax is:

```
var_name = input("prompt string")
```

At this point your program will stop and wait for input. It will continue waiting until the user types a newline (enter, carriage return, whatever you want to call it).

Everything the user types before the newline is stored in var\_name as a string

# Prompt strings

The prompt string in an input statement can only be a single string. You can't print out a paragraph telling the user what to do. You have to work around that.

## WRONG:

```
name = input("The program will now ask for your  
name.", "Please type it in the format lastname,  
firstname, middle initial, generational suffix.")
```

## RIGHT:

```
print("The program will now ask for your name.")  
print("Please type it in the format lastname,  
firstname, middle initial, generational suffix.")  
name = input("Enter your name now ")
```